KYNARA

Integration Guide

Put Kynara in front of the AI tools you already use — Claude Code, Claude Desktop, Cursor, Codex, and any MCP client — plus the SDK path for agents you build yourself.

Kynara · v1.0 · June 2026

CONTENTS

1 How Kynara fits in

Kynara governs the AI tools you already use — Claude Code, Claude Desktop, Cursor, Codex, and any MCP client — by sitting **in front of your MCP server** as a gateway. You point the tool at Kynara instead of directly at the MCP server; Kynara authorizes every tool call before it runs, hides tools an agent isn't allowed to use, and records every decision.

```
Your AI tool  ──────▶  Kynara MCP Gateway  ──────▶  Your MCP server
(Claude/Cursor/Codex)     allow / deny / approve        (CRM, Slack, files, …)
                          + least-privilege discovery
                          + tamper-evident audit
```

- **No agent code changes** — you only change one URL in the tool's MCP config.
- **Least-privilege discovery** — a tool only sees the MCP tools its agent identity may call.
- **Enforced outside the model** — a prompt injection can't talk its way past the gateway.

> **NOTE** One wrapper fronts one upstream MCP server. To govern several MCP servers, run one wrapper per server (different ports) and register each in Kynara.

2 Prerequisites

- A Kynara account (cloud at kynaraai.com, or self-hosted) with an **API key** (Settings → API Keys, scope `decisions.check`).
- A registered **agent identity** with a role that grants the scopes it needs (see the Setup Guide).
- The upstream **MCP server URL** you want to govern (e.g. your CRM or files MCP server).
- **Node 18+** (for the `mcp-remote` bridge used by Claude Desktop and Codex) and **Python 3.10+** or Docker to run the wrapper.

3 Step 1 — Register the MCP server in Kynara

1. Open **Enforcement → MCP Gateway → Register server**.
2. Enter a name, transport (`sse`), the upstream MCP URL, a scope prefix (e.g. `mcp.crm`), and a fail mode (`closed` = deny when the engine is unreachable — the secure default).
3. Copy the generated **Server ID** — you'll pass it to the wrapper as `KYNARA_MCP_SERVER_ID`.

> **TIP** You'll map the discovered tools to scopes in Step 3, after the wrapper syncs them.

4 Step 2 — Run the Kynara MCP wrapper

The wrapper connects to your upstream MCP server, discovers its tools, and serves them to your AI tool at `http://<host>:9090/sse` — enforcing Kynara policy on every call.

Python

```
git clone https://github.com/kynara/kynara-mcp-wrapper && cd kynara-mcp-wrapper
pip install -r requirements.txt

export KYNARA_UPSTREAM_MCP_URL="https://your-mcp-server.example.com/sse"
export KYNARA_API_KEY="sk_live_..."
export KYNARA_MCP_SERVER_ID="<server-id-from-step-1>"
```

```
python main.py        # serves http://localhost:9090/sse
```

Docker

```
docker run -p 9090:9090 \
  -e KYNARA_UPSTREAM_MCP_URL="https://your-mcp-server.example.com/sse" \
  -e KYNARA_API_KEY="sk_live_..." \
  -e KYNARA_MCP_SERVER_ID="<server-id>" \
  kynara/mcp-wrapper:latest
```

1. Check it's healthy: `curl http://localhost:9090/health` should report the upstream is reachable and how many tools were cached.
2. On startup the wrapper syncs the upstream tools back to Kynara for scope mapping.

5 Step 3 — Map tools to scopes & set least-privilege

1. Back in **MCP Gateway** → **[your server]**, confirm the discovered tools now appear.
2. Each tool auto-maps to a Kynara scope (e.g. `mcp.crm.contacts.read`). Adjust the scope, risk, or pin a tool to **deny** if it should never run.
3. Make sure your agent's role grants the scopes for the tools it legitimately needs — and nothing more.
4. Use **Least-privilege preview**: enter the agent ID and confirm only the intended tools are listed.

> **NOTE** Least-privilege discovery is an optimisation, not the security boundary — every call is still authorized per-policy even if a tool is invoked by name.

6 Step 4a — Claude Code

Add the gateway as an SSE MCP server. Pass the agent identity either as a URL query parameter (simplest, works everywhere) or as a header.

```
# agent id via query param (simplest)
claude mcp add --transport sse kynara-crm \
  "http://localhost:9090/sse?agent_id=<AGENT_ID>"

# …or via a header
claude mcp add --transport sse kynara-crm http://localhost:9090/sse \
  --header "x-kynara-agent: <AGENT_ID>"
```

1. Run `claude mcp list` to confirm `kynara-crm` is connected.
2. In a session, the tools you see are exactly the ones the agent is allowed to use.

> **IMPORTANT** All options (`--transport`, `--header`) must come *before* the server name. SSE is a legacy transport — if Kynara later exposes streamable HTTP, switch to `--transport http`.

7 Step 4b — Claude Desktop

Claude Desktop reaches remote/SSE servers through the `mcp-remote` bridge. Edit your config file:

- macOS: `~/Library/Application Support/Claude/claude_desktop_config.json`
- Windows: `%APPDATA%\Claude\claude_desktop_config.json`

```
{
  "mcpServers": {
    "kynara-crm": {
      "command": "npx",
      "args": [
        "mcp-remote",
        "http://localhost:9090/sse?agent_id=<AGENT_ID>"
      ]
    }
  }
}
```

1. Save the file and **fully restart** Claude Desktop.
2. Open the tools (plug) menu and confirm the Kynara-governed tools appear.

> **TIP** For a cloud-hosted gateway with OAuth, you can instead add it via Claude Desktop's **Custom Connectors** UI rather than the config file.

8 Step 4c — Cursor

Create `.cursor/mcp.json` in your project (or `~/.cursor/mcp.json` for all projects):

```
{
  "mcpServers": {
    "kynara-crm": {
      "url": "http://localhost:9090/sse",
      "headers": { "x-kynara-agent": "<AGENT_ID>" }
    }
  }
}
```

1. Open **Cursor Settings** → **MCP** and confirm `kynara-crm` shows as connected with a green status.
2. Cursor reconnects automatically if the wrapper restarts.

9 Step 4d — Codex CLI

Codex reaches the SSE gateway through the `mcp-remote` bridge. Add it via the CLI or `~/.codex/config.toml`:

```
# CLI
codex mcp add kynara_crm -- \
  npx mcp-remote "http://localhost:9090/sse?agent_id=<AGENT_ID>"

# …or ~/.codex/config.toml
[mcp_servers.kynara_crm]
command = "npx"
args = ["mcp-remote", "http://localhost:9090/sse?agent_id=<AGENT_ID>"]
```

1. In the Codex TUI, run `/mcp` to confirm `kynara_crm` is active.
2. A project-scoped `.codex/config.toml` requires the directory to be trusted.

10 Step 4e — Any other MCP client

Any MCP-compatible client works the same way — point it at the gateway and pass the agent id:

- **Native SSE/HTTP clients:** use the URL `http://localhost:9090/sse?agent_id=<AGENT_ID>` (or send an `x-kynara-agent` header).
- **stdio-only clients:** bridge with `npx mcp-remote http://localhost:9090/sse?agent_id=<AGENT_ID>`.

> NOTE The wrapper reads the agent id from the `x-kynara-agent` header, an `x-agent-id` header, or the `?agent_id=` query parameter. If none is supplied, the caller is treated as `anonymous` and least-privilege filtering applies.

11 Step 5 — Verify it works

1. **Least-privilege:** in your tool, list the available tools — only the ones the agent may call should appear (a pinned/denied tool is hidden).
2. **Allow:** invoke an allowed tool — it runs and returns a result.
3. **Deny:** invoke a tool the agent lacks scope for — you get a structured `[Kynara] Permission denied` error, and the action never reached the upstream server.
4. **Audit:** open Kynara's **Audit log** and confirm each call is recorded with the agent, tool, and outcome; run **Verify Chain**.

> ✓ **Pass:** Allowed tools work; denied/hidden tools are blocked; every call appears in the tamper-evident audit log.

12 Policies in action

Argument-level allowlists

Policies can read a tool call's arguments via `ctx.resource.attrs.<key>` — so you enforce *intent*, not just the action. Example: only let a Slack tool post to approved channels, or only let an email tool send to your domain.

```
{ "op": "in",
  "args": ["ctx.resource.attrs.channel", ["C0123", "C0456"]] }
```

Dynamic downgrade on untrusted input

The gateway automatically tracks **session taint**: when the agent calls a tool that returns untrusted content (a web fetch, an inbound email), later egress calls carry `context.taint`. A policy using `is_tainted` then denies or requires approval for those calls — so a prompt injection hidden in fetched content can't trigger data exfiltration. Load the *Block data egress after untrusted input* template to enable it.

> TIP Configure which tools taint output with `KYNARA_TAINT_SOURCE_TOOLS` (comma-separated name substrings); defaults cover web/fetch/read/email-style tools.

13 Governing agents you build yourself (SDK)

For agents you write in code (LangChain, LangGraph, CrewAI, AutoGen, OpenAI, Anthropic) rather than run through an MCP client, use the Kynara SDK to gate tools directly.

```
pip install kynara-sdk

from kynara_sdk import Kynara, permission_required
from kynara_sdk.context import set_current_kynara
```

```
set_current_kynara(Kynara(api_key=KEY, agent_id="crm-assistant",
                          user_id=user, fail_closed=True))

@permission_required("crm.contacts.read", resource_arg="contact_id")
def read_contact(contact_id):
    return crm.get(contact_id)   # only runs if Kynara allows
```

- **LangChain / LangGraph:** add `KynaraCallbackHandler` to your executor — it checks policy on `on_tool_start`.
- On `deny` the SDK raises `PermissionDenied` before the tool body runs; on `require_approval` it raises `ApprovalRequired` so the agent can pause.
- Same policies, roles, approvals, and audit as the gateway path — pick per agent.

14 Troubleshooting

Symptom	Likely cause & fix
No tools appear	Agent is `anonymous` or lacks scopes. Pass `?agent_id=` /header, and grant the agent's role the needed scopes.
Everything is denied	Fail-closed + engine unreachable, or missing role grants. Check `KYNARA_API_KEY`, the wrapper's `/health`, and the agent's Access Summary.
Connection refused	Wrapper not running or wrong port. Confirm `python main.py` is up and the URL/port match.
Header not honored	Some bridges drop custom headers. Use the `?agent_id=` query parameter instead.
Client won't connect	SSE vs HTTP mismatch. The wrapper serves SSE — use `--transport sse` / `mcp-remote`.

15 Reference — wrapper environment variables

Variable	Purpose
KYNARA_UPSTREAM_MCP_URL	The real MCP server the wrapper fronts.
KYNARA_API_KEY	Kynara API key (scope decisions.check).
KYNARA_MCP_SERVER_ID	Server ID from Step 1 — enables central scope mapping + least-privilege.
KYNARA_MCP_WRAPPER_PORT	Port to serve on (default 9090).
KYNARA_AGENT_ID_HEADER	Header the wrapper reads the agent id from (default x-kynara-agent).
KYNARA_TAINT_TRACKING	Enable session taint tracking (default true).
KYNARA_TAINT_SOURCE_TOOLS	Comma-separated tool-name substrings that taint output.
KYNARA_FAIL_OPEN	Global fallback when the engine is unreachable (default false = fail-closed).
KYNARA_SIDECAR_URL	Local sidecar for sub-ms decisions (tried before the central API).

Full docs: kynaraai.com/docs • MCP Gateway & wrapper README in the `kynara-mcp-wrapper` repo.